CHAPTER 5: TESTING, VALIDATION AND PERFORMANCE EVALUATION

5.1 TESTING STRATEGY

The testing of the IoT-Based Bus Stop System for Public Transport Overcrowding Prediction Using Machine Learning was planned at different levels to verify individual modules and their interaction under controlled operating conditions. Unit testing checked distance sensing, crossing confirmation, temperature acquisition, weather mapping, risk classification, operator correction and CSV writing. Integration and system testing then verified the complete flow from the ESP32 through Blynk Cloud to the local Python backend, dashboard, LEDs and Telegram alerts.

Both manual and tool-assisted testing were used. Hardware readings were checked using the Arduino IDE Serial Monitor, while backend logic and API responses were examined through the Python console and bus_stop_log.csv. The test environment combined an ESP32 prototype with brain.py running on a Windows machine, using simulated crowding conditions for Dindoshi Bus Depot.

The testing sequence followed a progressive approach: Unit Testing → Integration Testing → System Testing → Security Testing → Performance Evaluation → User Acceptance Testing. This order helped identify faults in separate components before assessing the complete prototype. The results in this chapter describe the documented prototype tests and their operating conditions.

5.2 UNIT TESTING

Unit testing was performed on individual hardware modules and software functions before complete integration. The VL53L0X was checked for two consecutive detections below 400 mm and rejection of an isolated low reading. The DHT22 was tested through GPIO4, and weather conditions were mapped to the binary rain feature used by the backend. Further tests checked the risk formula, the strict classification boundary at 75, the V7 passenger-count reduction and the six-field CSV record. For each test, the expected output was compared with the recorded actual output. These checks established the baseline behaviour required before connecting the sensing, cloud, prediction and output stages.

Test ID	Function / Module	Input	Expected Output	Actual Output	Pass/Fail
UT-01	VL53L0X crossing filter	Two readings below 400 mm	One crossing confirmed; count +1	One crossing confirmed; count +1	Pass
UT-02	VL53L0X noise rejection	Only one reading below 400 mm	Reject reading; count unchanged	Count unchanged	Pass
UT-03	DHT22 temperature	DHT22 data through GPIO4	Temperature read by ESP32	Temperature read correctly	Pass
UT-04	Weather mapping	Rain in weather API	Rain flag set to 1	Rain flag = 1	Pass
UT-05	Weather mapping	Clear weather response	Rain flag set to 0	Rain flag = 0	Pass
UT-06	Risk formula	45 passengers; no bonuses	Risk score = 75.0	Risk = 75.0	Pass
UT-07	Classification boundary	Risk exactly 75.0	WARNING status	WARNING	Pass
UT-08	Classification boundary	Risk score 76.0	CRITICAL status	CRITICAL	Pass
UT-09	V7 subtraction	Count 50; activate V7	Count reduced to 20	Count = 20	Pass
UT-10	V7 floor protection	Count 20; activate V7	Count reduced to 0	Count = 0	Pass
UT-11	CSV writer	One completed processing cycle	Append timestamp, passengers, temp, weather, risk and status	Complete CSV row appended	Pass

Table 5.2: Unit Testing

Testing Observations -

The VL53L0X and DHT22 modules were checked separately to confirm meaningful distance and temperature readings. Initial distance tests exposed a library and API mismatch. Rewriting the sensor routine with the Adafruit VL53L0X function set corrected the problem, after which the crossing and noise-rejection tests passed.

A second fault was traced to the breadboard supply wire being connected to the ESP32 VN input instead of the 3V3 output. Moving the wire restored sensor power. Software tests then confirmed the weather flags, the WARNING result at exactly 75.0, the CRITICAL result at 76.0, the V7 floor at zero and complete CSV records.

5.3 INTEGRATION TESTING

Integration testing checked the transfer of data between connected modules after unit testing. The tests focused on the ESP32, Blynk virtual pins, brain.py, OpenWeatherMap and Telegram. Particular attention was given to return communication from the Python risk calculation to the physical indicators and to preventing conflicting writes to shared cloud data.

Test ID	Modules Involved	Scenario	Expected Result	Actual Result	Status
IT-01	ESP32 + Blynk + brain.py	ESP32 publishes the updated count on V1	Backend reads the new count	Updated count read correctly	Pass
IT-02	brain.py + Blynk V3 + ESP32	Backend publishes a CRITICAL risk value	Red LED activates on GPIO25	Red LED activated	Pass
IT-03	brain.py + Telegram Bot API	Risk exceeds 75; repeat during cooldown	Alert delivered; 60 s cooldown enforced	Alert received; cooldown respected	Pass

IT-04	V7 + Blynk + ESP32	Operator presses Bus Left	Apply max(0, count - 30)	Count updated correctly	Pass
IT-05	OpenWeatherMap + brain.py	Fetch weather for Dindoshi, 19.1753, 72.8649	Map condition to the correct rain flag	Correct rain flag produced	Pass
IT-06	ESP32 + brain.py + Blynk pins	Edge writes V1/V4/V8; backend writes V2/V3/V5/V6	Stable values without overwrite conflicts	Strict pin ownership maintained	Pass

Table 5.3: Integration Testing

Integration Issues and Corrections -

Early integration exposed a virtual-pin overwrite conflict: the Arduino firmware and brain.py both attempted to write to V2 and V3. The cloud values changed unpredictably because the two programs did not have separate responsibilities. This was corrected by assigning each data item to a single publishing component.

The ESP32 was assigned V1 for passenger count, V4 for temperature and V8 for minutes since the last confirmed crossing. The Python backend was assigned V2 for weather, V3 for risk, V5 for date and time, and V6 for status. The V7 Bus Left command remained an operator input handled by the ESP32, which updated its own passenger count after applying the subtraction rule.

The output path was checked by publishing a CRITICAL risk value, receiving it at the ESP32 and observing the red LED on GPIO25. Telegram delivery was checked separately, including suppression of repeated notifications during the 60-second cooldown. These checks established that cloud values were connected to the intended physical and remote outputs.

The weather response was mapped to the rain feature before prediction. The backend combined this flag with hour, passenger count and temperature, then published weather, risk, time and status on its assigned pins. The same completed cycle supplied the local CSV record, linking the displayed result with the data retained for later review.

The final integration check followed a confirmed crossing through the entire pipeline. Sensor values reached Blynk, the backend obtained weather and time context, and the resulting risk and status were returned to the dashboard and indicators. Figure 5.3 shows these connected stages, including the CSV record and operator return path.

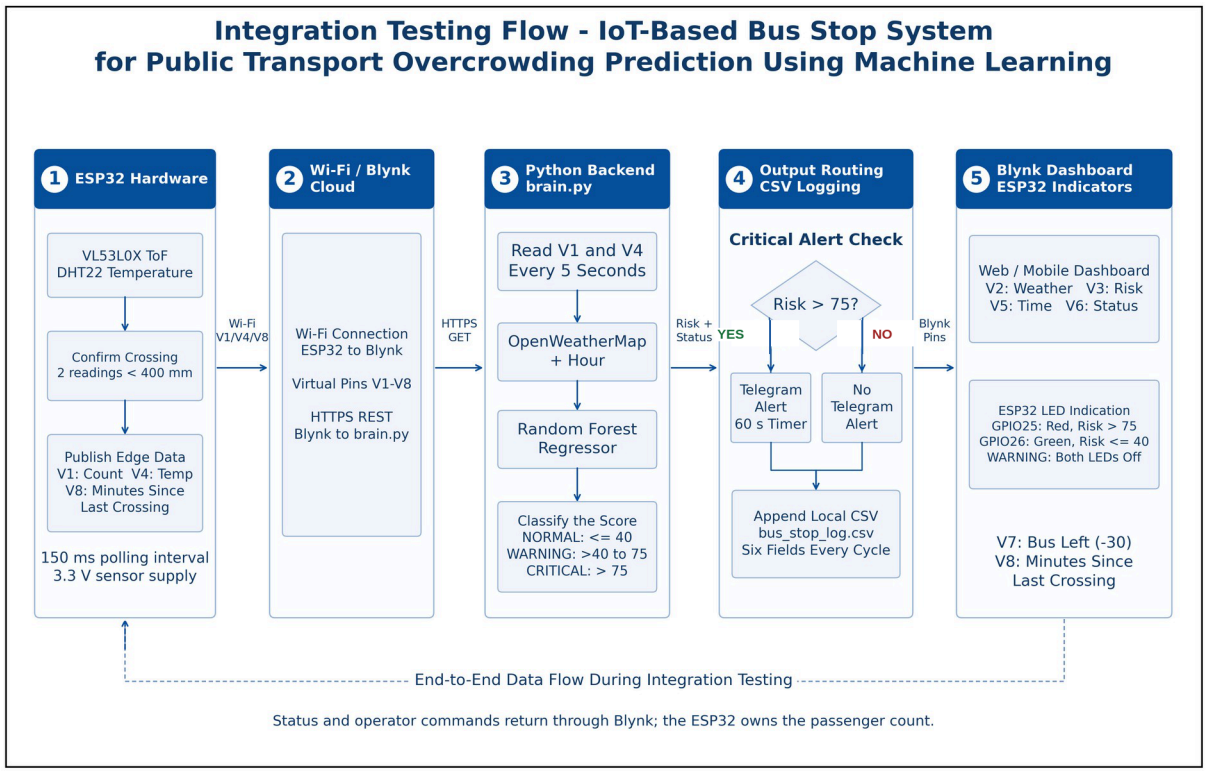

Figure 5.3: Integration Testing

5.4 SYSTEM TESTING

System testing assessed the complete bus stop prototype against the functional and non-functional requirements defined in Chapter 2. The full path from sensor input to Blynk, local prediction, dashboard display, LED indication, Telegram notification and CSV storage was exercised under simulated crowding conditions.

The tests covered crossing confirmation, weather enrichment, risk classification, the V7 correction command and silent public-facing operation. Table 5.4 summarizes the requirement checks recorded for the prototype.

Requirement ID	Requirement	Tested?	Met?
FR-1	Crossing Confirmation - Register one crossing after two consecutive distance readings below 400 mm.	Yes	Pass
FR-2	Temperature Reading - Read DHT22 temperature through GPIO4.	Yes	Pass
FR-3	Edge Publishing - Publish count, temperature and minutes since crossing on V1, V4 and V8.	Yes	Pass
FR-4	Weather Acquisition - Fetch Dindoshi weather and map it to a binary rain flag.	Yes	Pass
FR-5	Risk Computation - Run RandomForestRegressor using the four input features.	Yes	Pass
FR-6	Classification - NORMAL at or below 40; WARNING above 40 through 75; CRITICAL above 75.	Yes	Pass
FR-7	Critical Alerting - Activate the red LED and Telegram alert, with a 60-second cooldown.	Yes	Pass
FR-8	Operator Correction - V7 applies max(0, passenger count - 30).	Yes	Pass
FR-9	CSV Logging - Append a complete record after every processing cycle.	Yes	Pass
FR-10	Audible Alarm Prevention - Keep the wired buzzer disabled in firmware.	Yes	Pass
NFR-1	Dashboard Response - Update the live Blynk dashboard; reported latency was 891 ms.	Yes	Pass
NFR-2	Public Safety - Provide visual indication without an audible public alarm.	Yes	Pass
NFR-3	Usability - Display readable green, amber and red risk zones.	Yes	Pass
NFR-4	Risk Transparency - Use the documented risk calculation and a maximum score of 98.	Yes	Pass

Table 5.4: System Testing

The recorded end-to-end scenarios produced the intended output states: a reported risk of 90.0 activated the red LED and Telegram, 65.0 remained WARNING without an alert, and 15.0 produced NORMAL with the green LED. The V7 test reduced a count of 50 to 20. Exactly 75.0 remained WARNING.

The 90.0 and 65.0 values correspond to count-plus-rain calculations without heat or rush-hour bonuses. The scenario record also specifies 17:30, so its rush-hour flag must be checked before claiming exact formula agreement. The tests establish the reported output behaviour; future crowding prediction requires separate validation against field observations.

5.5 USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing assessed the clarity of the indicators, risk thresholds, dashboard and operator controls. The project record describes 25 participants: 10 commuters, five transit/depot staff, five depot administrators and five field technicians. Academic feedback was also obtained from the project guide. The recorded feedback and resulting refinements are summarized below.

Tester	Role	Task Given	Feedback	Action Taken
Project Guide	Academic Evaluator	Review the formula and thresholds	Thresholds should be transparent and defensible.	Documented capacity, bonuses and the strict >75 rule.
Commuter	End User	Interpret crowding status at a glance	Immediate visual indication was required.	Finalized red/green LEDs and dashboard colour zones.
Transit Staff	Operator	Use the V7 Bus Left control	Evidence was needed before deducting passengers.	Added the V8 minutes-since-crossing display.
Depot Admin	Reviewer	Check retained reading history	Persistent records were needed for review.	Verified CSV append after each processing cycle.

Table 5.5: User Acceptance Testing

UAT Outcome -

The feedback emphasized immediate visual awareness, understandable thresholds and controlled passenger-count correction. Existing feedback summaries report 84.0% preference for an on-site indicator and 92.0% expectation of an alert within 30 seconds. Question-specific response counts are not included in the record, so these percentages are not treated as additional measured UAT pass rates.

5.6 PERFORMANCE EVALUATION

Performance evaluation considered sensor timing, local model execution, dashboard updates, notification delivery and CSV writing. The values below reproduce the timings recorded during prototype testing. Measurements were obtained through firmware timers, Python timing functions, dashboard observation and the Telegram bot interface.

Performance Analysis -

The VL53L0X was polled every 150 ms, within the specified 200 ms limit. The confirmation filter required two consecutive readings below 400 mm, reducing the effect of an isolated distance drop. This polling interval is a firmware setting and is distinct from the complete sensor-to-alert response time.

The Random Forest predict() operation was reported at 10.51 ms, below the 100 ms target. Local inference therefore accounted for a small portion of the processing delay compared with cloud communication and the five-second backend polling schedule.

The Blynk gauge update latency was reported as 891 ms, against a five-second target. Telegram delivery was recorded as 0.81 seconds, below the ten-second target, while a 60-second cooldown prevented repeated alerts. These figures describe the measured stages and do not guarantee the same delay on every network connection.

The backend used a nominal five-second processing cycle. CSV writing was displayed as 0.00 ms by the timer; this is a value rounded to the recorded precision, rather than evidence of zero execution time. The complete cycle also includes API requests and processing, so its actual wall-clock duration depends on the loop implementation and network response.

Metric	Tool Used	Measured Result	Acceptable Target
VL53L0X Sensor Polling Interval	ESP32 firmware timer	150 ms	At most 200 ms
brain.py Processing Cycle	Python loop timer	5.0 seconds	At most 5.0 s
Blynk Gauge Update Latency	Dashboard observation	891 ms	Less than 5.0 s
Telegram Alert Delivery	Telegram bot chat	0.81 seconds	Less than 10.0 s
Telegram Alert Cooldown	Backend cooldown timer	60 seconds	Exactly 60 s
CSV Append Per Cycle	File-write timing	0.00 ms*	Less than 1.0 s
Random Forest Risk Computation	predict() function timing	10.51 ms	Less than 100 ms

Table 5.6: Performance Evaluation

Figure 5.6 compares four recorded timing values with their target limits using a common millisecond scale. The backend remained operational during the reported continuous cycles, while CSV rows retained the six required fields. *The displayed 0.00 ms CSV result is limited by timer precision.

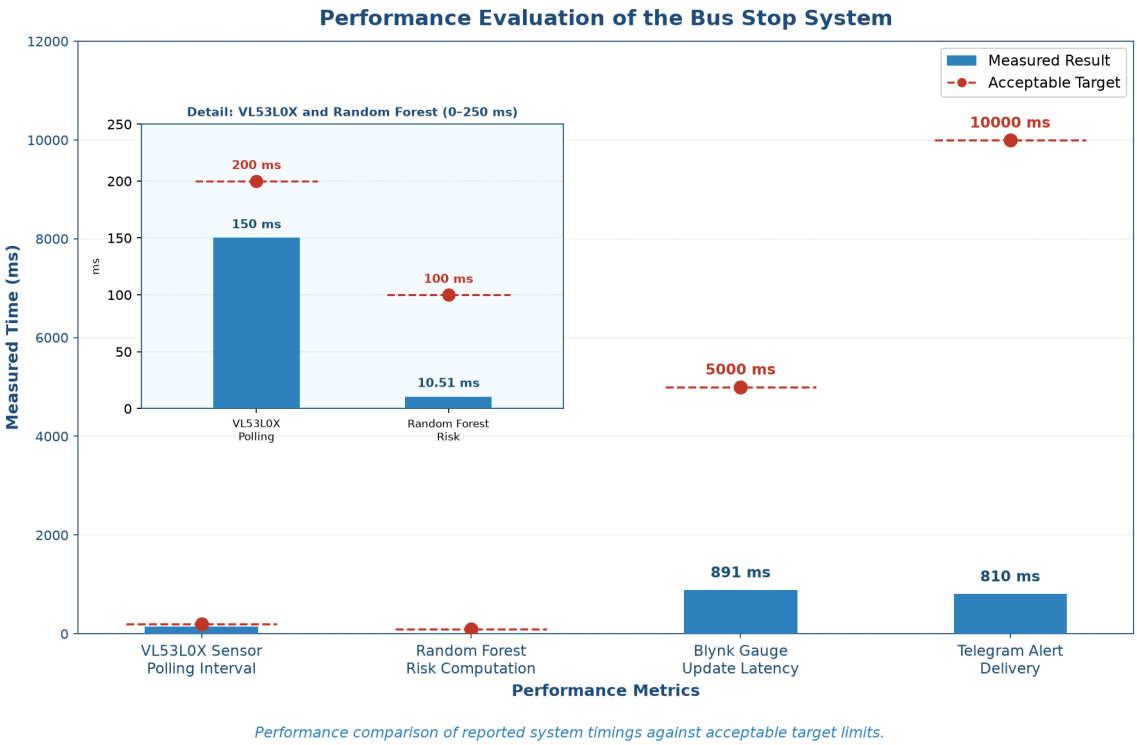

Figure 5.6: Performance Evaluation

5.7 SECURITY TESTING

Security and safety testing examined credential handling, dashboard access, external API transport, passenger privacy and public-facing outputs. The checks addressed the bus stop prototype and its actual interfaces: Blynk Cloud, OpenWeatherMap, Telegram, the ESP32 and the local Python files.

Submitted code examples and documentation were reviewed for exposed credentials. Blynk account access was checked, and external REST requests were inspected for HTTPS use. The privacy review considered whether the distance-sensing pipeline collected identifying passenger data. The physical-output check examined the behaviour of the system during CRITICAL status.

Test	Method	Result	Fix / Mitigation Applied
Token Exposure Risk	Review code examples and documentation	Tokens replaced with placeholders	Credential rotation and restricted local-folder access.
Dashboard Hijacking	Check Blynk access requirements	Account authentication required	Restrict dashboard access to authorized Blynk users.
API Transport Security	Inspect external REST API connection methods	HTTPS used for REST requests	Use HTTPS/TLS for Blynk REST, weather and Telegram.
Passenger Privacy	Review VL53L0X outputs and logged fields	Distance readings; no faces or device IDs	Use non-imaging sensing and aggregate count records.
Public Panic Hazard	Observe physical outputs during CRITICAL status	Buzzer identified as an audible-alarm risk	Keep the buzzer firmware-disabled; use LEDs and Telegram.

Table 5.7: Security Testing

The recorded checks confirmed the intended credential controls, authenticated dashboard access, HTTPS REST communication and non-imaging sensing. The buzzer remained disabled. These checks support prototype-level safety and access control, but do not constitute a full penetration test of the cloud services or a security certification.

5.8 VALIDATION METRICS

Validation summarized the successful executions recorded for unit tests, integration tests, boundary classification, critical alerts and continuous logging. The percentages measure agreement with the expected outcomes of these defined checks. They do not measure the ability to forecast actual future passenger demand.

1. Unit Test Pass Rate

The unit-test rate compares passed unit tests with all unit tests executed. The record contains 11 successful tests and no failures after the identified corrections.

$$Unit\ \ Test\ \ Pass\ \ Rate\ = \frac{Passed\ \ Unit\ \ Tests}{Total\ \ Unit\ \ Tests} \times\ 100$$

The recorded rate was 11/11 × 100 = 100.0%, exceeding the 90% target.

2. Integration Test Pass Rate

Integration integrity was calculated from successful data-transfer and output checks. All six recorded integration tests passed after pin ownership was enforced.

$$Integration\ \ Pass\ \ Rate\ = \frac{Passed\ \ Integration\ \ Tests}{Total\ \ Integration\ \ Tests} \times\ 100$$

The recorded rate was 6/6 × 100 = 100.0%, exceeding the 90% target.

3. Boundary Classification Agreement

Boundary testing recorded 15 successful trials out of 15, giving 100.0% agreement against a 95% target. The critical rule remained strictly above 75, with exactly 75.0 classified as WARNING.

4. Critical Alert and CSV Logging Success

The critical-alert record contains 20 successful trials out of 20, giving 100.0% success against a 95% target. Continuous logging produced 50 complete records over 50 cycles, meeting the 100% target. Each record contained timestamp, passengers, temperature, weather, risk and status.

Validation Metrics Summary -

Metric	Method / Formula	Result	Target	Met?
Unit Test Pass Rate	Passed unit tests / total × 100	100.0% (11/11)	>= 90%	Yes
Integration Integrity	Passed integration tests / total × 100	100.0% (6/6)	>= 90%	Yes
Boundary Classification	Correct classifications / trials × 100	100.0% (15/15)	>= 95%	Yes
Critical Alert Trigger	Successful alerts / trials × 100	100.0% (20/20)	>= 95%	Yes
Continuous CSV Logging	Complete records / cycles × 100	100.0% (50/50)	100.0%	Yes

Table 5.8: Validation Metrics

All five recorded indicators reached their stated targets. The Random Forest was trained on formula-generated data, so a successful threshold test should not be described as independently validated forecasting accuracy. A held-out error measure and comparison with observed crowding would be needed to assess prediction performance beyond these prototype checks.

5.9 TEST CASES

This section consolidates the important test cases used to verify the bus stop system. The selected cases cover confirmed and rejected crossings, risk boundaries, critical outputs, Telegram cooldown, operator correction and continuous CSV logging.

TC ID	Description	Pre-conditions & Steps	Expected Result	Actual Result	Status
TC-01	Confirm passenger crossing	Supply two consecutive readings below 400 mm.	One crossing; count +1.	One crossing; count increased.	Pass
TC-02	Reject an unconfirmed event	Supply a single reading below 400 mm only.	No crossing recorded.	Reading rejected.	Pass
TC-03	Full-capacity boundary	Set count to 45; all bonuses absent.	Risk 75.0; WARNING.	Risk 75.0; WARNING.	Pass
TC-04	CRITICAL output path	Run the recorded case: 45 passengers, rain, 17:30, 30°C.	Risk >75; red LED and Telegram.	Risk 90.0 reported; LED and alert active.	Pass
TC-05	Telegram cooldown	Repeat a CRITICAL event within 60 s.	Suppress the repeated alert.	Only one message during cooldown.	Pass
TC-06	V7 subtraction	Set count to 50 and press V7.	Count becomes 20.	Count became 20.	Pass
TC-07	V7 floor protection	Set count to 20 and press V7.	Count becomes 0.	Count became 0.	Pass
TC-08	Continuous CSV logging	Run three completed backend cycles.	Three complete rows appended.	Three rows appended.	Pass

Table 5.9: Test Cases

The selected cases cover the main operating path and were marked as Pass in the test record. For TC-04, the exact numerical risk should be reconciled with the rush-hour flag as explained in Section 5.4; the reported critical-output path remains clear.

5.10 DEFECTS IDENTIFIED AND CORRECTIONS

Testing identified faults in sensor communication, peripheral power and virtual-pin ownership. Each issue was corrected and the affected functions were retested before the final prototype checks.

Defect ID	Description	Severity	Fix Applied	Status
DEF-01	VL53L0X library/API mismatch caused sensor failure.	Critical	Rewrote the sensor routine using the Adafruit VL53L0X function set.	Resolved
DEF-02	Breadboard supply wire connected to VN instead of 3V3.	Major	Moved the supply wire to the regulated ESP32 3V3 output.	Resolved
DEF-03	Arduino and Python both wrote to V2 and V3.	Major	Assigned ESP32 V1/V4/V8 and Python V2/V3/V5/V6.	Resolved

Table 5.10: Defects Identified and Corrections

After these corrections, sensor readings stabilized and cloud values remained under the control of their assigned publisher. Retesting confirmed the intended count updates, risk classifications and output actions.

Remaining limitations include the inability of one VL53L0X to determine entry versus exit, the estimated 45-person capacity and 30-passenger departure correction, and dependence on Wi-Fi for live cloud updates and remote alerts. The model uses synthetic formula-generated training data, so field forecasting accuracy remains to be evaluated.

5.11 CHAPTER SUMMARY

This chapter evaluated the bus stop prototype through unit, integration, system, security, performance and user acceptance testing. The recorded checks confirmed two-frame crossing detection, the strict risk threshold above 75, V7 floor protection, separate virtual-pin ownership and complete CSV logging. Reported timings included 150 ms sensor polling, 891 ms dashboard updates, 0.81-second Telegram delivery and 10.51 ms Random Forest computation. The documented unit, integration, boundary, alert and logging checks achieved their stated pass-rate targets. User feedback supported clear indicators and the V7/V8 operator workflow, while the remaining limitations define the work needed for stronger field validation. Chapter 6 presents Community Deployment and Impact Assessment.